

CLASS 1

AI coding for VFX artists

Companion Guide. Everything from the class, written out, plus the prompts, a glossary and answers to common questions.

John Huikku · Claude Code · Codex · Antigravity · Windows, macOS and Linux

How to use this guide

You don't need to read this cover to cover. Read chapters 1 to 4 before your first session, then use the rest as a reference.

The class folder this came in (ai-coding-for-vfx-class) also has:

File	What it's for
00 START HERE	One page: what's in the folder, first three steps
01 Class 1 Slides	The deck from the class
03 Setup Guide	Installing your tool and the basics, on Windows, Mac or Linux
04 Safety & Your Data	One page worth keeping open
05 Stack Guide	What to use when you start building tools and apps
AGENTS.md , course.md , skills	Open the whole folder in your assistant and it becomes your personal tutor, with the studio rules and skills loaded
prompts/	Setup prompts and the prompt library
starter-pack/	Rules, skills and prompts you copy into your own project folders

The one idea to remember: you don't need to know *how*. You need to know *what to ask*, and how to check the answer. You already do that every day in dailies.

Contents

1. The big idea: giving notes
2. What these tools are
3. Pick your tool
4. Your first 15 minutes
5. Five safety habits
6. Brief it like you'd note a shot
7. When you're stuck
8. What you can do today
9. GitHub, in VFX terms
10. Where to go next

Appendix A: Prompt library · Appendix B: Glossary · Appendix C: FAQ

1. The big idea: giving notes

Most of the class is about tools, but the skill underneath is one you already have. Working with an AI coding assistant is the same loop as working with a junior artist:

- 1. **Brief:** say what you want, where, and what "done" looks like.
- 2. **Review:** look at what came back, carefully.
- 3. **Note:** say specifically what's wrong, and go around again.

The assistant writes the code. You direct it. Nobody in this class needs to become a programmer. You'll pick up a little along the way, and the assistant will explain whatever you ask about.

Why bother? The boring parts of the job (renaming 400 renders, making review copies, building a contact sheet, writing the same Nuke script again) take minutes instead of an hour. And you can build small tools for yourself and your team that nobody would ever have had time to make.

2. What these tools are

A chat app like ChatGPT answers in a window, and you copy and paste. An **AI coding assistant** (also called an *agent*) works **directly in your files and apps**:

It reads	It acts	You approve
Your files and folders, clip specs, error messages, screenshots	Renames, converts, writes scripts, runs ffmpeg, drives apps through MCP	It asks before it touches anything. You review the result like a dailies note.

Think of it as a fast junior artist who never gets tired and still needs supervising.

The brain and the hands

This is the most common worry, so here's exactly what happens when you say "convert this movie to H.264":

- 1. Your request goes to the **AI model in the cloud: the brain**. It plans the job and writes a script or command.
- 2. That command comes back and **runs on your computer: the hands**. ffmpeg (a free video tool) does the actual conversion, locally.
- 3. The **movie file never leaves your machine**. The only thing that came back from the cloud was one line of text, like:

```
ffmpeg -i clip.mov -vf "zscale=t=linear,tonemap=hable,zscale=t=bt709" -c:v libx264
clip_review.mp4
```

What does go to the cloud: your prompt, file names and paths, the text output of commands it runs (like a clip's specs), text files it opens, and **any frame or screenshot it looks at**. So:

- Keep confidential details out of prompts and file names.
- If it offers to "check a frame" of your footage, that frame is uploaded. When it matters, say: *"Don't view any frames. Check the specs only."*

3. Pick your tool

We recommend three. The ideas in this guide are the same in all of them, so **use the one your subscription already includes**, and start in the **desktop app**.

Tool	Comes with	Desktop app	Editor	Terminal (CLI)
Claude Code	Claude Pro, Max or Team	Claude app, Code tab	VS Code extension	Claude Code CLI
Codex	Any ChatGPT plan, including Free	ChatGPT app, Codex mode	VS Code extension	Codex CLI
Antigravity	Free tier, or Google AI Pro	Antigravity app	Antigravity IDE	Antigravity CLI

- **Desktop app:** a chat window pointed at a folder. Everyone starts here.
- **Editor:** the same assistant inside a code editor, for when you're keeping your own scripts and tools.
- **Terminal (CLI):** for automation and power users. Scheduled jobs and batch pipelines run this way.

Want to try it for free? Codex (ChatGPT Free) and Antigravity (free tier) both work without paying. Free tiers have lower usage limits.

Plans change often. This table was checked in September 2026. If something doesn't match, ask your assistant to check the current plans page.

Other names you'll hear

- **Chat apps:** ChatGPT, Claude, Gemini. Great for questions; you copy and paste the results yourself.
- **Other agents:** GitHub Copilot, OpenCode. Same idea as ours.
- **Editors with AI built in:** Cursor, Windsurf. Good tools; this class focuses on the three above, and the same ideas carry over.

4. Your first 15 minutes

The **Setup Guide** has the details for your operating system. This is the short version.

- 1. **Install your desktop app and sign in.** Claude (claude.com/download), ChatGPT (chatgpt.com/download, then switch to Codex) or Antigravity (antigravity.google/download).
- 2. **Get the class folder onto your own drive.** Clone or download **ai-coding-for-vfx-class** and put it in `Documents`. Not in Google Drive, OneDrive or Dropbox: synced folders can make everything crawl.
- 3. **Open that folder in the app.** The assistant can only work inside the folder you give it. That's its sandbox. This one comes with rules that make the assistant your class tutor, so it will introduce itself and ask a few questions.
- 4. **Ask it to make practice material, so you never experiment on real footage:**

Make a "practice" folder here with five short test clips using ffmpeg test patterns, and an image sequence of 48 frames starting at 1001. Give some of them messy names with spaces and mixed version numbers. Install ffmpeg first if it's missing (ask me before installing).

- 5. **Watch for the approval prompt.** Before it runs a command or installs something, it asks. Read what it's about to do. If you don't understand it, ask: *"Explain that command before I approve it."*
- 6. **Do your first real task, which only reads files:**

List every clip and image sequence in the practice folder with its duration, resolution and frame rate. Treat numbered frames as one sequence. Don't change anything.

- 7. **Review, then give a note.** Is the table right? Anything missing? Tell it. That's the whole loop.

Tip: you can paste screenshots into the chat (an error, a reference image, a sketch of a tool you want) and most apps let you dictate by voice. Both are often faster than typing.

5. Five safety habits

#	Habit	In practice
1	Work on copies	Point it at a copy or a test folder until you trust the task.

#	Habit	In practice
2	Dry run first	"Show me what you would change before you move, rename or delete anything."
3	Read before you approve	Keep approvals on. Don't switch on auto-approve until you're comfortable.
4	Watch what you paste	No passwords, API keys or confidential project details in prompts or file names.
5	Only trusted add-ons	MCPs and skills run code on your machine. Install them only from sources you trust.

Never delete, never overwrite. The starter-pack rules tell the assistant to move unwanted files into a `_trash/` folder (you empty it yourself) and to write outputs to a new folder like `_converted/`. It also keeps an **undo record** for batch renames, so "undo the last rename" just works.

Hidden instructions. Web pages, downloaded repos and documents can contain text aimed at the assistant ("ignore your instructions and..."). Be wary when you point it at things from the internet, and read approval prompts.

6. Brief it like you'd note a shot

Vague

```
convert these videos
```

Which videos? To what? Where do they go? Can it overwrite the originals?

Clear

```
In the review_copies folder, convert every
.mov to H.264 .mp4 at 1920x1080. Keep the
original file names. Show me the plan
first, and don't delete anything.
```

A good brief does four things:

- **Says where:** which folder, which files.
- **Says what done looks like:** format, size, names, where outputs go.
- **Gives an example** when it helps: "like `ABC_010_comp_v003.exr`".
- **Asks for the plan first** on anything bigger than a quick question.

7. When you're stuck

Ask it to teach you. After this class, the assistant is your teacher. House rule: **ask your AI before you ask a person.** It's faster, it's always there, and every answer teaches you something. These phrases unlock it:

- *"Explain this like I'm an artist, not a coder."*
- *"Ask me questions before you start."*
- *"What does this error mean? What do I do?"*
- *"Teach me as you go."*
- *"What should I learn next to do this myself?"*

Make it prove it. It can say "done" when it isn't. Ask it to show the file, list the outputs with their specs, play the result or open the page. Same as checking a render before it goes to dailies.

When it goes sideways:

- Start a fresh session. Long sessions get muddled.
- Give it more context: the folder, the app and version, the full error.
- Break the task into smaller steps.
- Try another model or tool.

Two things to know about how it thinks:

- **It forgets between sessions.** Anything it should always know (your naming convention, your rules) goes in an `AGENTS.md` file in the folder. The starter pack gives you one.
- **Its knowledge is a bit out of date.** For anything that changes fast (app versions, plans, prices, APIs) tell it: *"Check the current docs first."*

Your personal tutor. Open the class folder itself in your assistant and it switches into mentor mode: it asks your level, teaches one step at a time, makes practice files and keeps a learning log so it remembers your progress next time. Using a chat app instead? Paste `mentor-prompt.txt` into a new chat.

8. What you can do today

Organize

- **Inventory** a folder: specs of every clip, sequences detected, saved as a spreadsheet. Read-only, zero risk. The perfect first task.
- **Rename** to convention: `SHOW_SHOT_v###`, frame numbers kept intact. It shows a dry-run table of old → new names first.
- **Sort** a messy downloads folder into plates, references, renders, other.
- **Check** sequences for missing or duplicate frames.

Convert

Review copies, ProRes for editorial, proxies, image sequence → MP4, contact sheets, GIFs for chat, audio pulled from video. All of it runs on ffmpeg, locally.

The HDR lesson. We asked for an H.264 review copy of an iPhone clip. It came back washed out and too bright, because phones shoot **HDR** and a plain conversion ignores that. The note *"This looks washed out. It's HDR phone footage. Tone-map it to normal video."* fixed it in one pass. You don't need to know what tone-mapping is. You need to notice it looks wrong and say so. (The starter-pack `video-convert` skill now checks for HDR automatically.)

EXR sequences are linear, so a quick MP4 of them looks dark and contrasty without a color transform. The assistant can make an sRGB preview. Treat it as a preview, not show color.

Scripts for your apps

This works in every app, today, with no special setup: ask for a script, paste it into the app's script editor, run it.

App	Language	Where to paste it
Blender	Python	Scripting workspace → Text Editor → Run Script
Maya	Python	Script Editor → Python tab
Nuke	Python	Script Editor panel
Houdini	Python (VEX for geometry)	Python Shell, or a new Shelf Tool
Cinema 4D	Python	Extensions → Script Manager
3ds Max	Python or MAXScript	Scripting → Run Script / MAXScript Listener

App	Language	Where to paste it
After Effects	ExtendScript (.jsx)	File → Scripts → Run Script File
Photoshop	ExtendScript (.jsx) or UXP	File → Scripts → Browse
Premiere Pro	Depends on version	Ask the assistant what your version supports
DaVinci Resolve	Python or Lua	Workspace → Console (some scripting needs Studio)

Good habits the starter-pack `dcc-scripts` skill builds in: it asks your app version, acts on the selection, wraps changes in **one undo step**, and adds a `DRY_RUN` switch for anything that renames or deletes. If a script errors, paste the full error back and say "fix it."

Real example: a copper shader from one prompt. "Write a Maya Python script that builds a copper material for Arnold from scratch and assigns it to the selection." The script it wrote built the entire shading network (base color, metalness, roughness, all wired up) and it looked just like copper. No hand-wiring nodes. Then you iterate like a lookdev review: "more worn", "add a subtle patina in the crevices", "break up the roughness with a noise". Save the script in `scripts/maya/` and your whole team has a copper preset.

Next level: Nuke, Maya, Houdini and 3ds Max ship **PySide** (Qt), so you can ask for a tool panel with buttons for your most-used tasks.

One script, two ways: in the app or headless

The same Python file can run two ways:

- **In the app:** paste it into the script editor and run it. One scene, and you watch it happen. Best while you're trying and tweaking.
- **Headless (no window):** run it from the command line and the app never opens. Best once the script works: fix every shader in 50 scenes, build scenes from scratch, render a folder of files, or run it on another machine overnight.

Blender is the easiest place to start (tested with Blender 4.2 LTS):

```
# build a scene from scratch and save it
blender -b --python build_copper_scene.py -- out/copper.blend

# open an existing file and run a script on it
blender -b shot_010.blend --python fix_materials.py

# render frames 1001-1100 of a file (set -o before -a)
blender -b shot_010.blend -o //renders/shot_010.#### -s 1001 -e 1100 -a
```

`-b` means background (no window). Anything after `--` is passed to your script. The `.blend` file comes before `--python`.

Most DCCs have a headless mode:

App	Run a script without opening the app
Maya	<code>mayapy my_script.py</code> (start the script with <code>maya.standalone.initialize()</code>)
Houdini	<code>hython my_script.py</code>
Nuke	<code>nuke -t my_script.py</code>
Blender	<code>blender -b scene.blend --python my_script.py</code>
Cinema 4D	<code>c4dpy my_script.py</code>
3ds Max	<code>3dsmaxbatch my_script.ms</code> (MAXScript or Python)
After Effects	<code>aerender</code> for batch renders; scripts run inside the app
Photoshop, Premiere	No headless mode; a script loops over files while the app is open
Resolve	External scripting (Resolve Studio) drives the running app

You don't need to remember these. Ask:

Make this Blender script run headless on every .blend in <FOLDER>: open each file, run it, save the result as a new version (never overwrite), and give me a report. Do a dry run on 2 files first.

Commercial apps (Maya, Houdini, Nuke...) still use a license in headless mode, like opening the app normally. Blender is free, so it's the easiest place to experiment.

MCP: a plug that lets it drive your apps

MCP (Model Context Protocol) is a plug-in that gives the assistant hands inside another app: Blender, a web browser, GitHub, a database.

- **Without MCP:** it writes a script, you paste and run it.
- **With MCP:** it works in the app directly, building the scene, setting the lights, rendering.

All three tools support MCP. Each tool has its own settings, so you add an MCP once per tool (the setup prompts do it for you). **MCP for Blender** (formerly called Blender-MCP) is the one we use. Setup prompt 5 in the class folder walks you through installing it.

Safety: an MCP runs code on your machine. Only install ones from sources you trust, and use safe modes where they exist.

9. GitHub, in VFX terms

GitHub is where code lives, with every version kept. The main reason to use it: **git is the undo button for anything the assistant changes.**

Term	What it is	VFX version
Repo	A project folder that remembers every change	Show folder with every version
Commit	A save point, with a note on what changed	Versioning up v003 → v004
Diff	Exactly what changed between two versions	Before/after, for text
Push	Send your commits up to GitHub	Publishing to the server
Pull	Get the latest changes from GitHub	Syncing to latest
Clone	Download a repo, full history included	Copying down the whole project
Branch	A side version for trying an idea	A wip you can throw away
Merge	Bring a branch back into the main version	Promoting the wip to hero
Pull request	"Please review this and merge it"	Submitting for dailies
Fork	Your own copy of someone else's repo	Starting from a studio template
README	The repo's front page	The show bible
.gitignore	A list of files git must never save	"Never publish caches or renders"

You don't need the commands. Once the assistant is connected to GitHub (Setup Guide, step 6):

Commit this with a short note and push it to my private repo.

Make a branch called try-new-layout so we can experiment safely.

Undo what you did in the last commit.

Private by default

Never put in a repo: passwords, API keys, tokens · client or project data · people's names and emails · internal paths, sheet IDs, Drive links · footage, renders, EXRs (media stays on Drive; the starter-pack `.gitignore` blocks it).

Deleting isn't erasing. Git history keeps old versions. If a secret was ever pushed, treat it as leaked and replace it.

Go public only after a cleanup, on purpose. Ask the assistant to check every file *and the history* for secrets, names and internal links first.

Two special files: `.gitignore` and `.env`

- `.gitignore` lists what git must never save: footage, renders, caches, scene files, keys. The starter pack gives you one. To add something, ask: "Add my renders folder to `.gitignore`." It only stops *new* files; anything already committed stays in the history.
- `.env` is where your API keys live, one per line (`FAL_KEY=...`). `.gitignore` keeps it off GitHub. **Don't let the assistant open it:** a key it reads goes to the AI model, same as pasting it into chat. It only needs the key's *name*.

10. Where to go next

Which stack for what

When you want to build something, this tells you (and your assistant) where to start. The **Stack Guide** explains each piece.

Building...	Use
A quick utility: rename, convert, organize	Python + uv + ffmpeg
A utility with a simple UI	Python + Gradio
A tool inside a DCC	Python + PySide6, or an MCP
A desktop app for your team	Rust + Tauri with the web UI stack, or PySide6 + PyInstaller
A web app	React · Vite · Tailwind · shadcn, hosted on GitHub Pages, Netlify or Vercel
...that uses an API key	+ a serverless function or a Railway backend
...that needs logins or a database	+ Supabase

Building...	Use
...that stores lots of media	+ Backblaze B2 or Cloudflare R2

Front end, back end, full stack. The **frontend** is the part of an app people see and click, running in their browser. The **backend** is the private part on a server: it holds the keys, talks to the database and does the heavy work. **Full stack** means building both halves (plus the database) as one project, which your assistant can do for you.

One rule to remember now: API keys (the passwords that let your tool use a paid service) never go in a web page or a desktop app, because anyone can look inside those. They live on a backend. **Frontend asks, backend spends.**

The starter pack

Copy `starter-pack/` into any project folder (Setup Guide, step 7) and your assistant follows studio rules every time:

- `AGENTS.md`: the rules: dry run first, never delete, naming, codecs. Read by Claude Code, Codex, Antigravity and Cursor. Edit the naming and delivery sections for your show.
- **Skills:** safe file operations, video conversion (with the HDR fix), contact sheets, DCC scripts.
- `.gitignore`: keeps media, caches and keys out of GitHub.
- `prompts/`: the setup prompts and the prompt library (Appendix A), copy-paste clean.

Your first week

Pick one:

1. List the specs of every clip in a folder (read-only)
2. Rename a render folder to convention (dry run first)
3. Turn an image sequence into an MP4
4. Make a contact sheet of a shot folder
5. A script for your main app's most boring task
6. A Gradio app that runs a fal.ai image tool
7. Put it all in a private GitHub repo

Two kinds of cost: your subscription has usage limits (they reset every few hours or weekly); services you call through an API, like fal.ai, bill separately. Set a spending limit on every API account.

Stuck? Ask in this order:

1. **Your AI assistant.** Always first: paste the error, ask it to explain.
2. **The class folder**, opened in your assistant, for step-by-step help.
3. **These docs** and the prompt library.

4. **Ask John**, once you've tried 1 to 3. Tell him what you asked your AI and what happened. And send him your wins!

Appendix A: Prompt library

These are also in `starter-pack/prompts/prompt-library.md`. Copy from that file rather than this PDF, because PDFs can mangle quotes and line breaks.

AI Coding for VFX Artists · John Huikku

Copy, paste, edit the parts in `<ANGLE BRACKETS>`. Works in Claude Code, Codex and Antigravity. The starter-pack `AGENTS.md` already tells the assistant to dry-run, never delete and prove its work, but saying it again never hurts.

Getting unstuck & learning

Explain this like I'm a VFX artist, not a programmer.

Ask me questions before you start.

What does this error mean, and what should I do? `<PASTE ERROR>`

Teach me as you go: explain each step in one line before you do it.

What should I learn next so I can do this myself?

This isn't working. Stop, tell me what you've tried, and suggest a simpler way.

Before you start, check the current official docs for `<TOOL>`. Your knowledge might be out of date.

Prove it

Prove it's done: show me the output files, their specs, and the file counts before and after.

Check the outputs with `ffprobe` only. Don't open or view any frames.

Organize

Inventory (read-only, a perfect first task):

List every video and image sequence in <FOLDER> with its duration or frame range, resolution, frame rate and codec. Treat numbered frames as one sequence. Save it as a CSV and give me a summary. Don't change anything.

Rename to convention:

Rename the renders in <FOLDER> to <SHOW>_<SHOT>_v###.<ext>. Keep frame numbers intact. Show me a dry-run table of old → new names first and wait for my OK. Save an undo file.

Sort a messy folder:

Sort <FOLDER> into subfolders: plates, references, renders, audio, other. Show me the plan first. Move nothing until I approve. Never delete.

Find problems:

Check the image sequences in <FOLDER> for missing frames, duplicate frames and inconsistent resolutions. Report only, don't fix anything.

Tidy versions:

In <FOLDER>, find every shot that has more than 3 versions. Move all but the latest 3 into _trash/, after showing me the list and getting my OK.

Convert

Review copies:

Convert every .mov in <FOLDER> to an H.264 .mp4 review copy. Put them in _converted/. Check each clip first. If any are HDR phone footage, tone-map them to normal Rec.709. Prove it with ffprobe at the end.

ProRes for editorial:

Make ProRes 422 HQ copies of the clips in <FOLDER> for editorial, into _converted/. Keep the audio.

Image sequence → MP4:

Turn the image sequence in <FOLDER> into an MP4 at <24> fps. If it's EXR, make a quick sRGB preview and tell me it's a preview transform.

Proxies:

Make half-resolution H.264 proxies of everything in <FOLDER>, same names with _proxy on the end.

Contact sheet:

Make a contact sheet of every clip in <FOLDER>: middle frame of each, 5 per row, file name under each. Save it as a JPG.

GIF for chat:

Make a small GIF (640 wide, 12 fps) of <CLIP> from <00:02> to <00:06>.

Watch folder:

Write me a small tool that watches <FOLDER> and automatically makes an H.264 review copy of any new .mov that lands in it. Explain how to start and stop it.

DCC scripts

Paste the result into the app's script editor (the assistant will tell you where). Ask for `DRY_RUN = True` on anything that renames or deletes.

Blender

Write a Blender Python script that sets up a three-point light rig and a turntable camera orbiting the selected object over 120 frames.

Maya

Write a Maya Python script that renames the selected objects to <prefix>_### with padded numbers, as one undo step.

Maya: build a shader from scratch

Write a Maya Python script that builds a copper material for Arnold from scratch (aiStandardSurface, physically plausible copper color, metalness and roughness) and assigns it to the selected objects. Name the nodes clearly.

Nuke

Write a Nuke Python script that sets every selected Read node's frame range to the project range and reports any Read with missing files.

Houdini

Write a Houdini shelf tool (Python) that creates a camera and a basic three-light rig looking at the selected object.

Cinema 4D

Write a Cinema 4D Python script that puts every selected object under a new null named after the first object, with undo.

3ds Max

Write a 3ds Max Python (pymxs) script that lists every material in the scene and which objects use it, and saves the list to a text file.

After Effects

Write an After Effects script (.jsx) that lists every comp's name, size, duration and frame rate in a text file.

Photoshop

Write a Photoshop script that exports every layer group as its own PNG named after the group.

Premiere Pro

What scripting does Premiere Pro <VERSION> support? Then write a script that lists every clip in the active sequence with its in/out timecodes.

DaVinci Resolve

Write a DaVinci Resolve Python script (for Workspace → Console) that adds a marker with the clip name at the start of every clip on video track 1. Tell me if this needs Resolve Studio.

Blender headless: build scenes from scratch

Write a Blender Python script I can run headless (`blender -b --python`) that builds a turntable scene from scratch: a copper sphere, a three-point light rig, a camera orbiting over 120 frames, then saves it to the path I pass after `--`. Show me the exact command to run it.

Batch: run a script over many scenes

Turn this script into a batch tool: run it without opening <APP> (batch or headless mode) on every scene file in <FOLDER>, save each result as a new version (never overwrite), and write a report of what changed. Do a dry run on 2 files first.

Tool panel (Maya / Nuke / Houdini / 3ds Max)

Make me a PySide panel for <APP> with buttons for these three tasks: <TASK 1>, <TASK 2>, <TASK 3>. Explain how to install it so it's always there.

MCP (drive the app directly, once set up)

In Blender, add a 3-point light rig and a turntable camera around the selected object, then render a clay preview to <FOLDER>.

GitHub

Turn this folder into a private GitHub repo with the starter-pack `.gitignore` and push it. Explain each step in one line.

Commit this with a short note about what we changed, and push it.

Make a branch called <try-idea> so we can experiment without touching the working version.

Undo what you did in the last commit.

Before I make this repo public, check every file and the whole git history for keys, passwords, emails, names, internal paths and links. Report only.

Small apps & APIs

First Gradio tool:

Build a small local Gradio app with uv: I drop in a video, it shows the specs and gives me a button to make an H.264 review copy. Keep share off.

First API project:

Build a tiny web page that shows the weather for <CITY>, using a free weather API. Keep the API key on a small backend, not in the page. Tell me where to put the key; don't ask me to paste it here.

fal.ai image tool:

Build a local Gradio app that sends a prompt to fal.ai and shows the generated image. Read my FAL_KEY from a .env file. Before you start, check fal.ai's current docs for the model and Python client.

Web app starter:

Create a new web app with React, Vite, TypeScript, Tailwind, shadcn/ui and Lucide icons, in a private GitHub repo. Add a Railway backend that holds my API keys. Explain the folder structure in 5 lines.

Desktop app starter:

Create a Tauri desktop app (Rust core, web-style UI) with React, Vite, Tailwind and shadcn/ui that lists the videos in a folder I pick and shows their specs. Bundle ffmpeg with it.

Privacy

Do this without viewing any frames or images. Use file specs only.

Don't include file paths or names in anything you save to the repo.

Appendix B: Glossary

Term	Meaning
Agent / AI coding assistant	An AI that works in your files and apps (Claude Code, Codex, Antigravity), not just a chat window.
AI model	The "brain" in the cloud that plans and writes. The assistant app is how you talk to it.
Prompt	What you type (or say) to the assistant. A brief.
Session	One conversation. It forgets when you start a new one, unless the rules are in <code>AGENTS.md</code> .
AGENTS.md	A text file of rules in a folder that the assistant reads automatically.
Skill	A saved how-to the assistant loads when a task matches (e.g. <code>video-convert</code>).
MCP	A plug-in that lets the assistant control another app directly.
Dry run	Showing what <i>would</i> change, without changing anything.
Terminal / CLI	A text window for typing commands. The assistant uses it for you.
Script	A small program: a list of steps a computer (or an app like Nuke) runs.
Python	The scripting language most VFX apps use.
uv	A tool that installs Python and keeps each project's add-ons separate.
ffmpeg / ffprobe	Free tools that convert video (ffmpeg) and report its specs (ffprobe).
HDR	High dynamic range. Phone footage is often HDR and needs tone-mapping for normal screens.
Tone-mapping	Squeezing HDR brightness into normal (Rec.709) range so it looks right.
Rec.709	The standard color space for normal HD video.
Linear	Light values as rendered, before a display transform. EXRs are usually linear.
Gradio	A Python library that puts a simple web UI on a script.
PySide / Qt	The toolkit for tool panels in Maya, Nuke, Houdini and 3ds Max.
Tauri	Builds small, fast desktop apps: a window made like a web page, with a Rust core underneath.
Rust	A fast, safe programming language. Here it's the engine inside Tauri apps; your assistant writes it, you don't need to learn it.

Term	Meaning
Frontend / backend	The part of an app people see (frontend) and the private server part (backend).
Full stack	All of it together: frontend, backend and database. A "full-stack" project or developer covers both halves.
Headless / batch mode	Running a DCC's scripts without opening its window (<code>bLender -b</code> , <code>mayapy</code> , <code>hython</code>), to process many files or run on another machine.
API	A way for programs to use another service (weather, fal.ai image generation).
API key	The password that lets your tool use an API. Often billed. Keep it secret.
.env file	A local file where API keys live, kept out of git. Don't let the assistant open it.
.gitignore	A list of files git must never save (media, caches, keys).
Repo, commit, push...	See chapter 9.
Usage limit	How much you can use your subscription before it pauses until the next reset.
Token	Two meanings: a unit of AI usage (limits are counted in them), or a password-like key (keep secret).

Appendix C: FAQ

Do I need to learn to code? No. You need to brief clearly and check results. You'll pick up some along the way, and you can always ask it to explain.

Which tool should I pick? The one your subscription includes. Nothing yet? Codex and Antigravity both have free tiers. The skills are the same in all three.

Is my footage uploaded to the AI company? No. The model writes a script; your computer runs it. What goes up is text (your prompt, file names, command output) plus any frame or screenshot it looks at. Say "specs only, don't view frames" when it matters.

Can it break or delete my files? It can only work in the folder you give it, it asks before acting, and the starter-pack rules forbid deleting and overwriting. Keep approvals on, work on copies, and use git as your undo button.

It asked me to approve a command I don't understand. Don't approve it yet. Ask: *"Explain that command in plain words. What will it change?"*

It said it was done, but it isn't. That happens. Ask it to prove it: list the outputs, show the specs, open the result. Then give a note.

It forgot what we did yesterday. Each session starts fresh. Put lasting rules in `AGENTS.md`. The class folder keeps a learning log, so your tutor remembers where you left off.

I hit my usage limit. Limits reset every few hours (and some weekly). Meanwhile, use smaller, clearer requests. Long rambling sessions use more.

Can it work inside Nuke / Maya / Houdini directly? Today: it writes scripts you paste into the script editor. For Blender, MCP for Blender lets it drive the app directly. Other apps have community MCPs; check where they come from before installing.

Do I really need GitHub? Not on day one. But it's the undo button and the way to share tools, so set it up in your first week (Setup Guide, step 6).

What's the difference between ChatGPT and Codex? ChatGPT is the chat; Codex is the coding assistant that works in your files. They're in the same desktop app.

I'm on Linux / Mac / Windows. Does that matter? The assistant detects your system and uses the right commands. The Setup Guide covers all three.

Can I share a tool I made with my team? Yes: a private GitHub repo you invite them to, or ask the assistant to package it (PyInstaller for Python tools). Unsigned apps show a security warning on first launch; that's expected for in-house tools.

The answer seems out of date. It probably is. Say: *"Check the current official docs first."*